

Entropiq — План разработки

Версия: 1.0

Дата: 2026-02-16

Подход: Вертикальный (каждая фича: миграции → логика → UI → тесты)

Содержание

0. Фундамент и дизайн-система
 1. Аутентификация
 2. Операции (доходы/расходы)
 3. Финансовые цели
 4. Инфляция и покупательная способность
 5. Аналитика и графики
 6. AI-советы
 7. Freemium и подписки
 8. Профиль, настройки, уведомления
 9. Лендинг и публичные страницы
 10. Онбординг
 11. Seeder и демо-данные
 12. Тестирование и стабилизация
 13. Stage-деплой и CI/CD
 14. Подготовка к запуску и защите
-

Календарь

Период	Фаза	Статус
15–28 февраля	Фаза 0: Фундамент и дизайн-система	—
16–22 марта	Фаза 1: Аутентификация	—
23–31 марта	Фаза 2: Операции	—
1–10 апреля	Фаза 3: Финансовые цели	—
11–20 апреля	Фаза 4: Инфляция и покупательная способность	—
21–27 апреля	Фаза 5: Аналитика и графики	—
28–30 апреля	Фаза 12a: Внутреннее тестирование (первый проход)	—
1–10 мая	Альфа-тестирование (5–10 тестировщиков)	—
11–20 мая	Фаза 12b: Исправление багов и улучшения по фидбеку	—
21–28 мая	Фаза 6: AI-советы	—
29–31 мая	Фаза 7: Freemium и подписки	—
1–3 июня	Фаза 8: Профиль, настройки, уведомления	—
3–5 июня	Фаза 9: Лендинг и публичные страницы	—
5–6 июня	Фаза 10: Онбординг	—
параллельно	Фаза 11: Seeder и демо-данные	—
ближе к финалу	Фаза 13: Stage-деплой и CI/CD	—
26–30 июня	Фаза 14: Подготовка к защите	—

Даты ориентировочные. Фазы 7–10 компактнее, т.к. инфраструктурная база уже готова после фаз 1–6.

Фаза 0: Фундамент и дизайн-система

Срок: 15–28 февраля

Цель: Подготовить каркас приложения, все миграции, layout, навигацию и базовые UI-компоненты до начала разработки фич.

0.1. Проектирование БД (все миграции)

Создаются все миграции сразу — это даёт целостную картину данных и позволяет избежать ошибок связей на поздних этапах.

Таблицы:

Таблица	Назначение	Ключевые поля
users	Пользователи	phone, name, locale, currency_code, subscription_plan, onboarding_completed_at
verification_codes	SMS-коды OTP	user_phone, code, expires_at, attempts
personal_access_tokens	Sanctum-токены	(стандартная миграция)
currencies	Справочник валют	code (ISO 4217), name, symbol, decimal_places
categories	Категории операций	name, type (income/expense), icon, color, is_system, user_id (null = системная), sort_order
transactions	Операции	user_id, category_id, type (income/expense), amount (bigint, копейки), currency_code, date, comment, is_recurring, recurring_interval
recurring_rules	Правила повторяющихся операций	user_id, transaction_template_id, interval (daily/weekly/monthly/yearly), next_run_at, is_active
goals	Финансовые цели	user_id, name, type (enum), icon, target_amount, current_amount, currency_code, started_at, target_date
goal_contributions	Взносы на цели	goal_id, transaction_id (nullable), amount, date

сpi_values	Данные ИПЦ (Росстат)	period (date), category_code, value (decimal), source
сpi_categories	Справочник категорий ИПЦ	code, name, parent_code, mapping_to_app_category_id
ai_advices	AI-рекомендации	user_id, title, body, basis_data (json), rating (nullable), is_read, generated_at
subscriptions	Подписки пользователей	user_id, plan (free/monthly/yearly), status, starts_at, ends_at, payment_id
payments	История платежей	user_id, subscription_id, amount, currency_code, provider, provider_payment_id, status, paid_at
notification_settings	Настройки уведомлений	user_id, email_weekly, push_goals, push_ai_advice
activity_log	Аудит (Spatie)	(стандартная миграция пакета)

Enums (app/Enums/):

Enum	Значения
TransactionType	Income, Expense
GoalType	SafetyNet, LargePurchase, Travel, Car, Apartment, Education, Other
GoalStatus	Active, Achieved, Cancelled
RecurringInterval	Daily, Weekly, Monthly, Yearly
SubscriptionPlan	Free, Monthly, Yearly
SubscriptionStatus	Active, Cancelled, Expired
PaymentStatus	Pending, Completed, Failed, Refunded
Locale	Ru, En

Модели (app/Models/):

Все модели создаются вместе с миграциями. Каждая модель содержит:

- Typed properties
- Casts (enums, dates, money)
- Relationships
- Scopes (при необходимости)

Factories (database/factories/):

Для всех моделей — понадобятся для тестов и seeders.

0.2. Layout и навигация

Задача	Детали
Установка и конфигурация Flux	Подключение пакета, настройка Tailwind под проект
Цветовая схема	Настройка <code>tailwind.config.js</code> : primary (indigo), success (green), danger (red), warning (orange), premium (violet)
Типографика	Шрифт, размеры (H1–Small, Number), настройка в Tailwind
Guest layout	Минимальный layout для публичных страниц (лендинг, авторизация)
App layout (mobile)	Нижняя навигация (5 пунктов: Дашборд, Операции, +Добавить, Цели, Ещё)
App layout (desktop)	Левая боковая панель (expanded), те же пункты + подменю
Responsive переключение	< 640px → мобильная навигация, >= 1024px → десктопная

0.3. Базовые UI-компоненты

Blade-компоненты (`resources/views/components/`), построенные на Flux + Tailwind:

Компонент	Файл
EmptyState	empty-state.blade.php
PremiumLock	premium-lock.blade.php
BottomNav	bottom-nav.blade.php
MoneyDisplay	money-display.blade.php
PhoneInput	phone-input.blade.php
OtpInput	otp-input.blade.php
CategoryIcon	category-icon.blade.php

0.4. Роутинг (каркас)

```

routes/web.php      – публичные страницы + авторизованная зона
routes/api.php      – API v1 (для будущего мобильного приложения)
routes/console.php  – artisan-команды (импорт CLI, генерация советов)

```

Middleware:

- `auth` — стандартный Laravel
- `verified.phone` — проверка подтверждённого телефона (кастомный)
- `subscription` — проверка уровня подписки для premium-фич (кастомный)

0.5. Языковые файлы (каркас)

```

lang/ru/auth.php
lang/ru/transactions.php
lang/ru/goals.php
lang/ru/analytics.php
lang/ru/advice.php
lang/ru/subscription.php
lang/ru/common.php
lang/en/... (зеркальная структура)

```

0.6. Сервисы (каркас)

Пустые классы с интерфейсами — заполняются в соответствующих фазах:

Сервис	Назначение
SmsService	Отправка SMS (заглушка → SMS Aero)
InflationService	Расчёты инфляции, пересчёт покупательной способности
GoalCalculationService	Расчёт взносов, прогноз даты, сценарии
AiAdviceService	Генерация AI-рекомендаций (правила + LLM)
SubscriptionService	Управление подписками, проверка лимитов
ExportService	Экспорт PDF/Excel

Фаза 1: Аутентификация

Срок: 16–22 марта

Ветка: feature/auth

Зависимости: Фаза 0

1.1. Backend

Задача	Детали
Модель <code>User</code>	Поля: <code>phone</code> , <code>name</code> , <code>locale</code> , <code>currency_code</code> , <code>subscription_plan</code> , <code>onboarding_completed_at</code> . Casts: <code>locale</code> → <code>Locale</code> enum, <code>subscription_plan</code> → <code>SubscriptionPlan</code> enum.
Модель <code>VerificationCode</code>	Поля: <code>phone</code> , <code>code</code> (4 цифры), <code>expires_at</code> , <code>attempts</code> , <code>verified_at</code> . Scope: <code>active()</code> .
<code>SmsService</code> (заглушка)	Интерфейс <code>SmsServiceInterface</code> + реализация <code>LogSmsService</code> (код пишется в лог / отправляется в Telegram). Регистрация в <code>ServiceProvider</code> .
<code>AuthController</code> (web)	Методы: <code>showLogin</code> , <code>sendCode</code> , <code>verifyCode</code> , <code>logout</code> .
<code>Api\V1\AuthController</code>	Методы: <code>sendCode</code> , <code>verifyCode</code> (возвращает Sanctum-токен), <code>logout</code> .
<code>SendVerificationCode</code> (Action)	Генерация 4-значного кода, сохранение, вызов <code>SmsService</code> . Rate limiting: 1 SMS / 60 сек на номер.
<code>VerifyCode</code> (Action)	Проверка кода, создание/обновление пользователя, создание сессии. Max 3 попытки.
Form Requests	<code>SendCodeRequest</code> (валидация телефона), <code>VerifyCodeRequest</code> (валидация кода).
Rate limiting	<code>RateLimiter::for('sms', ...)</code> — 1 запрос/60 сек. <code>RateLimiter::for('verify', ...)</code> — 5 запросов/минуту.
Тесты	Отправка кода, верификация, повторная отправка, rate limiting, невалидный код, истёкший код.

1.2. Frontend

Задача	Детали
Экран ввода телефона	PhoneInput , кнопка "Получить код", юридический текст. Livewire-компонент.
Экран ввода SMS-кода	OtpInput (4 поля, auto-focus), таймер повторной отправки (60 сек), ссылка "Изменить номер". Livewire-компонент.
Обработка ошибок	Красная подсветка полей, текст ошибки, состояние disabled.
Редирект после входа	Новый пользователь → онбординг. Существующий → дашборд.

Фаза 2: Операции

Срок: 23–31 марта

Ветка: feature/transactions

Зависимости: Фаза 1

2.1. Backend

Задача	Детали
Модель <code>Category</code>	Системные категории (20 штук) — seeder. Поля: name (json для i18n), type, icon, color, is_system, user_id, sort_order.
Модель <code>Transaction</code>	Поля: user_id, category_id, type, amount (bigint), currency_code, date, comment, is_recurring. Scopes: forPeriod() , income() , expense() , byCategory() .
<code>TransactionsController</code>	CRUD: index (с фильтрацией, пагинацией), store, show, update, destroy.
<code>Api\V1\TransactionsController</code>	REST API зеркало web-контроллера.
<code>TransactionResource</code>	API-ресурс для форматирования ответа.
<code>StoreTransactionRequest</code>	Валидация: type (required, in:income,expense), amount (required, integer, min:1), category_id (required, exists), date (required, date), comment (nullable, string, max:255).
<code>TransactionService</code>	Методы: getForPeriod() , getSummary() (доходы/расходы/баланс за период), getByCategory() .
<code>CategorySeeder</code>	20 системных категорий (16 расходных + 4 доходных) с иконками и цветами.
Повторяющиеся операции	Модель <code>RecurringRule</code> . Artisan-команда recurring:process — в Scheduler ежедневно. Создаёт транзакции по правилам.
Тесты	CRUD операций, фильтрация, валидация, пагинация, summary, повторяющиеся операции.

2.2. Frontend

Задача	Детали
Список операций	Livewire-компонент. Группировка по дням. Фильтры: тип (tabs), период (dropdown), категория (multi-select). Поиск по комментариям. Сводка (доходы/расходы/баланс). Бесконечный скролл. Свайп влево → удалить (Alpine.js).
Форма добавления	Livewire-компонент. Переключатель Расход/Доход (tab). Поле суммы (крупное, числовая клавиатура). Сетка категорий 4×N (CategoryGrid). Дата (по умолчанию сегодня). Комментарий (скрыт, по нажатию). Повтор (скрыт).
Форма редактирования	Та же форма, предзаполненная данными.
Подтверждение удаления	Modal (Flux).
Пустое состояние	EmptyState компонент: "У вас пока нет операций".
Компонент TransactionRow	Иконка категории + текст + сумма (цвет по типу) + дата.
Дашборд (v1)	Первая версия: баланс (сумма доходов – расходов), 3 метрики (доходы/расходы/экономия за месяц), список последних 5 операций.

Фаза 3: Финансовые цели

Срок: 1–10 апреля

Ветка: feature/goals

Зависимости: Фаза 2

3.1. Backend

Задача	Детали
Модель Goal	Поля: user_id, name, type (GoalType enum), icon, target_amount, current_amount, currency_code, started_at, target_date, status (GoalStatus enum). Accessors: progress_percent, remaining_amount, is_achieved.
Модель GoalContribution	Поля: goal_id, transaction_id (nullable), amount, date. Связь с Transaction (опциональная).
GoalsController	CRUD + contribute (внесение взноса).
Api\V1\GoalsController	REST API зеркало.
GoalCalculationService	Методы:
	requiredMonthlyPayment(targetAmount, currentAmount, monthsLeft) — без инфляции
	requiredMonthlyPaymentWithInflation(targetAmount, currentAmount, monthsLeft, annualInflation) — с инфляцией
	predictCompletionDate(targetAmount, currentAmount, monthlyPayment, annualInflation) — прогноз даты
	buildScenarios(goal) — 3 сценария (оптимист 5%, базовый текущий%, пессимист 15%)
	whatIf(goal, additionalMonthly) — пересчёт при изменении взноса
StoreGoalRequest	Валидация: name, type, target_amount, target_date (after:today), initial_amount (nullable).
ContributeRequest	Валидация: amount (required, integer, min:1).
Тесты	CRUD, расчёт взносов, прогноз даты, сценарии, взнос на цель, достижение цели. Расчётные тесты: проверка формул из research.md.

3.2. Frontend

Задача	Детали
Список целей	Livewire-компонент. Карточки целей (GoalCard). Каждая: иконка + название + двойной прогресс-бар + "45 000 / 200 000 ₽" + прогноз даты.
Форма создания	Пошаговая форма (Livewire): тип (горизонтальный скролл карточек) → название → сумма + начальная сумма → срок (date picker или preset). Мгновенный расчёт необходимого взноса при изменении полей.
Детали цели	Livewire-компонент. Прогресс (двойной прогресс-бар). 4 карточки метрик. Сценарии (3 карточки). "Что если?" слайдер (Alpine.js). График план vs факт (ApexCharts). Список взносов. Кнопки: "Внести взнос", "Редактировать", "Удалить".
Компонент DualProgressBar	Два прогресс-бара: номинальный (синий) + реальный (оранжевый). Blade-компонент.
Компонент GoalCard	Blade-компонент для списка и дашборда.
Компонент ScenarioTable	3 сценария в карточках.
Компонент WhatIfSlider	Слайдер + мгновенный пересчёт (Alpine.js + Livewire wire:model.live).
Дашборд (v2)	Добавить: горизонтальная лента целей. Если целей нет — СТА "Создайте первую цель".
Пустое состояние	"Поставьте финансовую цель, и мы поможем её достичь!"

Фаза 4: Инфляция и покупательная способность

Срок: 11–20 апреля

Ветка: feature/inflation

Зависимости: Фаза 3

4.1. Backend

Задача	Детали
Модель <code>CpiValue</code>	Поля: <code>period</code> , <code>category_code</code> , <code>value</code> (decimal 8,4), <code>source</code> . Index: (<code>period</code> , <code>category_code</code>).
Модель <code>CpiCategory</code>	Поля: <code>code</code> , <code>name</code> , <code>parent_code</code> , <code>mapping_to_category_id</code> (nullable). Связь с <code>Category</code> приложения.
Artisan-команда <code>cpi:import</code>	Импорт данных ИПЦ из ЕМИСС (fedstat.ru). Для MVP — загрузка из подготовленного CSV/JSON-файла. Парсинг, валидация, <code>upsert</code> в <code>cpi_values</code> .
<code>InflationService</code>	Методы:
	<code>getCurrentCpi()</code> — текущий общий ИПЦ
	<code>getCpiForPeriod(from, to)</code> — ИПЦ за произвольный период
	<code>getCpiByCategory(categoryCode, period)</code> — ИПЦ по категории
	<code>calculateRealValue(nominalAmount, fromDate, toDate)</code> — пересчёт номинальной суммы в реальную
	<code>calculatePersonalInflation(userId, period)</code> — персональная инфляция на основе структуры расходов
	<code>calculateInflationLoss(userId, period)</code> — сумма, потерянная из-за инфляции
<code>CpiSeeder</code>	Предзаполнение реальными данными ИПЦ за последние 2–3 года (из открытых данных Росстата).
Обновление <code>GoalCalculationService</code>	Использовать реальные данные ИПЦ вместо захардкоженных процентов.
Тесты	Импорт CPI, расчёт реальной стоимости, персональная инфляция, потери от инфляции. Проверка формул из <code>research.md</code> на контрольных данных.

4.2. Frontend

Задача	Детали
Дашборд (v3)	Карточка баланса: номинальный (крупно) + реальный (мельче) + разница ("−43 379 Р из-за инфляции"). Мини-виджет персональной инфляции.
Компонент BalanceCard	Номинальный + реальный баланс, разница со стрелкой и цветом.
Компонент InflationWidget	"Ваша инфляция: 10,2% (средняя по РФ: 9,5%)".
Обновление целей	Двойной прогресс-бар теперь использует реальные данные инфляции. Сценарии — реальный ИПЦ.

Фаза 5: Аналитика и графики

Срок: 21–27 апреля
Ветка: feature/analytics
Зависимости: Фаза 4

5.1. Backend

Задача	Детали
AnalyticsService	Методы:
	<code>getExpensesByCategory(userId, period)</code> — расходы по категориям (для donut chart)
	<code>getBalanceDynamics(userId, period)</code> — баланс по дням/месяцам, номинальный + реальный
	<code>getPersonalInflationBreakdown(userId, period)</code> — категория + доля + ИПЦ + вклад
	<code>getTrends(userId, period)</code> — тренды по категориям (↑↓ vs предыдущий период)
Api\V1\AnalyticsController	Эндпоинты для 3 вкладок аналитики.
Тесты	Корректность расчётов агрегации, пустые данные, разные периоды.

5.2. Frontend

Задача	Детали
Экран аналитики (3 вкладки)	Livewire-компонент с Tabs (Alpine.js).
Вкладка "Расходы по категориям"	Donut chart (ApexCharts). Список категорий: иконка + название + сумма + % + тренд. Переключатель периода.
Вкладка "Динамика баланса"	Line chart (ApexCharts): 2 линии — номинальный (синий) + реальный (оранжевый). Tooltip при наведении. Переключатель периода. Текст: "Инфляция обесценила ваши накопления на X ₽".
Вкладка "Персональная инфляция"	Главная метрика (крупно). Таблица breakdown.
Компонент DonutChart	Blade-обёртка над ApexCharts. Props: data, labels, colors.
Компонент LineChart	Blade-обёртка над ApexCharts. Props: series (массив линий), categories (ось X).
Обновление навигации	Аналитика доступна из меню "Ещё".

Фаза 6: AI-советы

Срок: 21–28 мая
Ветка: feature/ai-advice
Зависимости: Фазы 2, 4, 5

6.1. Backend

Задача	Детали
Модель AiAdvice	Поля: user_id, title, body, basis_data (json), rating (nullable), is_read, generated_at. Scopes: unread() , recent() .
Rule Engine	AiAdviceRuleEngine — набор правил:
	Правило 1: Рост расходов в категории > 20% vs средняя за 3 мес
	Правило 2: Расходы > доходов за месяц
	Правило 3: Цель отстаёт от плана
	Правило 4: Крупная нетипичная транзакция
	Правило 5: Потенциальная экономия (рекомендация по оптимизации)
	Каждое правило: evaluate(User): ? AdvicePayload
LlmService (заглушка)	Интерфейс LlmServiceInterface . Реализация FakeLlmService — возвращает шаблонные тексты на основе данных из Rule Engine. Позже: GigaChatService , YandexGptService , OpenAiService .
AiAdviceService	generateForUser(User) — запускает Rule Engine, формирует payload, вызывает LLM для генерации текста совета.
Artisan-команда advice:generate	Генерация советов для всех пользователей. В Scheduler — ежедневно.
AiAdviceController	index (список), show (детали), rate (👍/👎).
Api\V1\AiAdviceController	REST API зеркало.
Тесты	Срабатывание каждого правила, генерация совета, рейтинг, корректность basis_data.

6.2. Frontend

Задача	Детали
Список AI-советов	Livewire-компонент. Карточки (AiAdviceCard): дата + заголовок + текст + "Подробнее".
Детали совета	Полный текст + блок "На чём основан совет" (метрики из basis_data). Кнопки 👍/👎 (Livewire).
Компонент AiAdviceCard	Blade-компонент. Вариант: полный / заблюренный (для free).
Дашборд (v4)	Добавить: карточка "Совет дня" (последний непрочитанный совет).
Пустое состояние	"Для первого совета нам нужны данные. Внесите операции за 1–2 недели".
Обновление навигации	AI-советы доступны из меню "Ещё".

Фаза 7: Freemium и подписки

Срок: 29–31 мая

Ветка: feature/subscriptions

Зависимости: Все предыдущие фазы

7.1. Backend

Задача	Детали
Модель Subscription	Поля: user_id, plan, status, starts_at, ends_at, payment_id.
Модель Payment	Поля: user_id, subscription_id, amount, currency_code, provider, provider_payment_id, status, paid_at.
SubscriptionService	Методы:
	isPremium(User): bool
	canCreateGoal(User): bool — проверка лимита (free: 1 цель)
	canAddTransaction(User): bool — проверка лимита (free: 50/мес)
	getRemainingTransactions(User): int
	canViewAdvice(User, AiAdvice): bool — free: 1/неделю
	canViewPeriod(User, period): bool — free: 1 месяц
	subscribe(User, plan): Subscription
	cancel(User): void
Middleware CheckSubscription	Для premium-only роутов. Возвращает 403 с информацией о необходимости подписки.
Платёжная заглушка	FakePaymentService — имитирует успешную оплату. Позже: YooKassaService .
Внедрение лимитов	Обновить контроллеры: Goals (лимит на количество), Transactions (лимит на количество/мес), Analytics (лимит на период), AiAdvice (лимит на частоту).
Тесты	Проверка лимитов free, переход на premium, снятие лимитов, истечение подписки, откат на free.

7.2. Frontend

Задача	Детали
Экран подписки	Сравнение Free vs Premium (таблица). Две карточки тарифов. Кнопка "Оформить". Гарантия возврата.
Компонент PremiumLock	Overlay с blur +  + CTA. Blade-компонент с slot.
Внедрение PremiumLock	Аналитика (расширенные периоды), сценарии целей, "Что если?", персональная инфляция, заблокированные AI-советы.
Баннеры upsell	Лимит операций (при 40+), попытка создать 2-ю цель, нажатие на заблокированный контент.

Фаза 8: Профиль, настройки, уведомления

Срок: 1–3 июня

Ветка: feature/profile

Зависимости: Фаза 7

8.1. Backend

Задача	Детали
ProfileController	Обновление имени, locale, currency.
NotificationSettingsController	CRUD настроек уведомлений.
Модель NotificationSetting	Поля: user_id, email_weekly, push_goals, push_ai_advice. Значения по умолчанию — все включены.
Email-уведомления	WeeklyDigestMail — еженедельная сводка (Mailable). В Scheduler: weekly() .
Экспорт данных	ExportService → CSV. Artisan-команда + web-эндпоинт.
Удаление аккаунта	Soft delete + анонимизация данных (152-ФЗ).
Тесты	Обновление профиля, настройки уведомлений, экспорт, удаление аккаунта.

8.2. Frontend

Задача	Детали
Экран профиля	Аватар (инициалы), имя, телефон. Кнопка "Выйти".
Настройки	Подписка (текущий план + СТА). Уведомления (toggles). Язык (select). Валюта (select). Данные (экспорт CSV, удаление аккаунта). О приложении (версия, ссылки).

Фаза 9: Лендинг и публичные страницы

Срок: 3–5 июня

Ветка: feature/landing

Зависимости: Фаза 4 (для калькулятора инфляции)

9.1. Backend

Задача	Детали
<code>InflationCalculatorController</code>	Принимает: сумму, дату. Возвращает: реальную стоимость, потерю. Использует <code>InflationService</code> . Без авторизации.

9.2. Frontend

Задача	Детали
Лендинг	Секции: Hero, Проблема, Калькулятор инфляции (интерактивный, Livewire), Возможности, Сравнение с конкурентами, Тарифы, Footer. Guest layout.
Калькулятор инфляции	Livewire-компонент. Ввод суммы + выбор даты (или preset). Мгновенный результат. СТА → регистрация.
Юридические страницы	Политика конфиденциальности, Пользовательское соглашение — статические Blade-шаблоны.

Фаза 10: Онбординг

Срок: 5–6 июня

Ветка: `feature/onboarding`

Зависимости: Фазы 2, 3

10.1. Backend

Задача	Детали
Флаг <code>onboarding_completed_at</code>	В модели <code>User</code> . Middleware <code>CheckOnboarding</code> — редирект на онбординг, если <code>null</code> .
<code>OnboardingController</code>	Методы: <code>step1</code> , <code>step2</code> , <code>step3</code> , <code>complete</code> .

10.2. Frontend

Задача	Детали
Экран 1: Приветствие	Иллюстрация + заголовок + текст + "Далее". Индикатор ● ○ ○.
Экран 2: Первая цель	"Создать цель" (→ форма) / "Позже". Индикатор ○ ● ○.
Экран 3: Первая операция	"Добавить операцию" (→ форма) / "Начать с чистого листа" (→ дашборд). Индикатор ○ ○ ●.
Компонент OnboardingSlide	Blade-компонент: иллюстрация + заголовок + текст + кнопки + индикатор.

Фаза 11: Seeder и демо-данные

Срок: параллельно с разработкой фич, финальная сборка перед альфа-тестом

Ветка: feature/seeders

Задача	Детали
DemoSeeder (главный)	Оркестрирует все остальные seeders. <code>php artisan db:seed --class=DemoSeeder</code> .
Демо- пользователь	Телефон: +7 (999) 000-00-01, имя: "Алексей", Premium-подписка.
Операции (6 месяцев)	300–400 транзакций за 6 месяцев. Реалистичное распределение: зарплата 2 раза/мес, продукты 8–12 раз/мес, ЖКХ 1 раз/мес, кафе 4–6 раз/мес, транспорт ежедневно. Случайный разброс сумм (±20%). Сезонные паттерны (больше на развлечения летом).
Цели (3 штуки)	"Отпуск в Турцию" (200 000 ₽, через 8 мес, прогресс 30%), "Подушка безопасности" (300 000 ₽, через 12 мес, прогресс 15%), "Новый ноутбук" (120 000 ₽, через 4 мес, прогресс 60%).
AI-советы (10 штук)	Реалистичные советы за последние 2 месяца. Разные типы правил. Часть прочитана, часть нет.
Данные ИПЦ	Реальные данные за 2024–2026 из Росстата.

Фаза 12: Тестирование и стабилизация

12a. Внутреннее тестирование (28–30 апреля)

Задача	Детали
Функциональное тестирование	Проход по всем user flows из ux-design.md
Тестирование на устройствах	Mobile (375px, 390px), Tablet (768px), Desktop (1280px, 1440px)
Исправление критических багов	—
Прогон Pest	Все тесты проходят
Прогон Larastan	Level 6, ноль ошибок
Прогон Pint	Код-стайл чистый

12b. Исправления по фидбеку альфа-теста (11–20 мая)

Задача	Детали
Баг-трекер	Сбор и приоритизация задач из фидбека
Исправление багов	По приоритету: critical → high → medium
Улучшения UX	Доработка на основе обратной связи
Подсказки и пояснения	Тултипы, тексты помощи

Фаза 13: Stage-деплой и CI/CD

Срок: ближе к финалу (конец мая — начало июня)

Ветка: feature/deploy

Задача	Детали
Покупка домена	Выбор и регистрация
Настройка VPS (Timeweb)	Установка Docker, Docker Compose, настройка firewall
Деплой на stage	docker-compose.stage.yml, настройка .env.stage
SSL (Let's Encrypt)	Certbot, автообновление
Nginx reverse proxy	Проксирование на Docker-контейнер
GitHub Actions CI	Прогон пайплайна: Pint → Larastan → Pest
GitHub Actions CD	Автодеплой на stage при мерже в stage
Envoy	Настройка deploy-скриптов
Мониторинг	Laravel Pulse, Sentry (если есть аккаунт)
Бэкапы	Cron + backup-db.sh

Фаза 14: Подготовка к запуску и защите

Срок: 26–30 июня

Задача	Детали
Финальный деплой	Production или демо на stage
Демо-данные	Прогон DemoSeeder для красивой демонстрации
Презентация	Структура: проблема → решение → демо → метрики → планы
Сценарий демонстрации	Пошаговый скрипт: вход → дашборд → операции → цели → инфляция → AI-совет
Сбор метрик	Количество пользователей, операций, конверсия (если запуск был раньше)
Репетиция	Прогон демонстрации на stage

Сквозные задачи

Задачи, которые выполняются на протяжении всей разработки, а не в отдельной фазе:

Задача	Когда
Мультиязычность (__())	При создании каждого текста в UI
Мультивалютность (bigint + currency_code)	При создании каждой денежной модели
API v1 контроллеры	Параллельно с web-контроллерами для каждой фичи
Тесты (Pest)	В конце каждой фазы
Код-стайл (Pint)	Перед каждым коммитом
Статический анализ (Larastan)	Перед каждым PR
Адаптивность	При вёрстке каждого экрана (mobile-first)
Spatie Activity Log	Для критичных операций (создание/удаление транзакций, целей, изменение подписки)
Git workflow	feature/* → PR → stage